

OpenZeppelin Sui Payments Audit



July 10, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Low Severity	9
L-01 Invoices and Vouchers Created with Wrong Parameters Cannot Be Canceled Before Expiration	9
L-02 Unbounded Variants per Listing Can Make Pricing and Removal Exceed Transaction Limits	10
L-03 Reused Voucher Preimages Enable Cross-Voucher Redemption Replay	10
Notes & Additional Information	11
N-01 Configuration and Display Events Omit the Values They Change	11
N-02 Documentation Styling Inconsistencies	12
N-03 Redundant and Duplicated Code in merchant Module	13
N-04 Incorrect Documentation	14
N-05 Incomplete Documentation	16
Conclusion	18
Appendix	19
Issue Classification	19

Summary

Type	Library	Total Issues	8 (8 resolved)
Timeline	From 2026-06-29 To 2026-07-03	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	3 (3 resolved)
		Notes & Additional Information	5 (5 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [OpenZeppelin/openzeppelin-sui-payments](#) repository at commit [a81cb25](#).

In scope were the following files:

```
contracts
├── payments
│   └── sources
│       ├── config.move
│       ├── events.move
│       ├── listing.move
│       ├── loyalty.move
│       ├── merchant.move
│       ├── payment.move
│       ├── receipt.move
│       └── redemption.move
```

System Overview

The [openzeppelin-sui-payments](#) package implements a closed-loop stablecoin payment, loyalty, and redemption system for point-of-sale flows on Sui. A merchant deploys the package to publish a catalog of priced items, issue invoices that customers settle in a single accepted stablecoin, reward each settlement by minting a soulbound [LOYALTY](#) currency, and honor vouchers through which customers later redeem accumulated loyalty for goods. Custody of both currencies is delegated to the Permissioned Asset Standard (PAS), an external dependency in which balances are held inside per-user [Account](#) objects and every movement is authorized against an issuer-configured [Policy](#). The package acts as the PAS issuer for [LOYALTY](#) and as a PAS consumer for the merchant's chosen stablecoin.

All protocol state is concentrated in a single shared [Merchant](#) object, which stores the listing catalog, open invoices, open vouchers, and settlement receipts in tables, alongside the merchant's [Config](#) and the [Loyalty](#) bundle that encapsulates the [TreasuryCap<LOYALTY>](#) and the PAS policy capabilities. The [merchant](#) module hosts the business logic, while the [payment](#), [redemption](#), [listing](#), and [receipt](#) modules define the [Invoice](#), [Voucher](#), [Listing](#), and [Receipt](#) value types stored inside the [Merchant](#). The [config](#) module validates and holds the merchant parameters, the [loyalty](#) module manages the loyalty currency and its PAS policy, and the [events](#) module emits the events consumed by off-chain indexers. Invoices and vouchers are store-only values keyed by freshly minted IDs that are surfaced off-chain (for example, as QR codes) for the counterparty to settle against.

Invoice Settlement: A cashier issues an invoice through [create_invoice](#) from catalog variants and quantities. The invoice snapshots the payout address, the accepted payment type, line-item prices, the total amount, the loyalty reward computed by [compute_loyalty](#), and an expiry bounded by a seven-day maximum time-to-live, so an issued invoice is an immutable commitment for its lifetime. The customer settles through one of two permissionless paths: [pay](#) resolves a customer-approved PAS [send_funds](#) request against the stablecoin's policy and moves the funds into the PAS account of the snapshotted payout address, whereas [pay_with_coin](#) accepts a plain [Coin](#) and transfers it directly to the payout address. Both paths mint the snapshotted loyalty into the customer's PAS account and store a [Receipt<Payment>](#) in the [Merchant](#). An expired invoice holds no funds and can be removed permissionlessly through [cancel_invoice](#).

Voucher Redemption: A customer creates a voucher permissionlessly through `create_voucher` by resolving a PAS `unlock_funds` request, which extracts a `Balance<LOYALTY>` equal to the priced items' total from the customer's account and locks it inside the voucher together with a 32-byte `blake2b256` commitment to a customer-chosen secret. Redemption through `redeem` is executed by a cashier and additionally requires the preimage of that commitment, after which the locked balance is burned and the loyalty supply decreased. Once a voucher expires, redemption is blocked and the permissionless `cancel_voucher` path refunds the locked balance into the customer's PAS account. The `LOYALTY` currency is [soulbound by construction](#): its PAS policy registers an approval only for the `unlock_funds` action consumed during voucher creation and none for account-to-account transfers, so loyalty can only be earned on settlement, locked into vouchers, burned on redemption, or refunded on cancellation.

Administrative authority is provided by a shared `AccessControl<MERCHANT>` registry from the OpenZeppelin Sui access control library, created at package publication with the deployer as the sole holder of the root role. The root role administers three operational roles, `MerchantRole`, `CatalogManagerRole`, and `CashierRole`, whose capabilities are described in the following section.

Security Model and Trust Assumptions

Root role holder: The deployer receives the root role of the shared `AccessControl<MERCHANT>` registry and is the default admin of every operational role, so it can grant and revoke `MerchantRole`, `CatalogManagerRole`, and `CashierRole` at any time. Root transfer and renouncement are subject to a 24-hour timelock. The root role controls the entire administrative surface and is trusted to be held securely, ideally separated from daily operations, and to grant operational roles only to intended addresses.

`MerchantRole` holders: These holders can atomically replace the merchant's entire configuration through `update_config`, covering the payout address, the accepted payment currency and its decimals, the loyalty coefficient, the per-payment loyalty cap, and both time-to-live values. They can also update the display name and logo, and [prune stored receipts](#), which irreversibly removes the receipts' line-item breakdown from chain state. Because every settlement-relevant field is snapshotted at issuance, configuration changes affect only future invoices and vouchers. The role is trusted to configure sensible economic parameters and, in

particular, to set the payout address to an address that is able to receive funds: [as documented](#), Sui cannot distinguish an externally owned address from an object ID on-chain, so a payout address pointing at an object routes settlements into transfer-to-object semantics under which the funds may be unrecoverable.

CatalogManagerRole holders: These holders manage the catalog by adding and removing listings and variants and toggling each listing's active status. Since invoice issuance prices line items based on the catalog contents at that moment, the role is trusted to keep stablecoin and loyalty prices correct. However, catalog edits never retroactively reprice open invoices or vouchers.

CashierRole holders: These holders issue invoices, whose snapshotted terms remain settleable until expiry, and execute voucher redemption. The hashlock is the only barrier between a cashier and the loyalty locked inside a voucher: [redeem](#) verifies that the [blake2b256](#) hash of the submitted preimage matches the voucher's stored 32-byte commitment, but it does not constrain the length or entropy of the preimage itself. Since every voucher's commitment is publicly readable on-chain, a malicious or compromised cashier could brute-force a low-entropy secret offline (for example, a short PIN) and redeem the voucher without the customer presenting it, permanently burning the locked balance. The hashlock is therefore only as strong as the entropy of the customer-chosen secret, and wallets and client integrations are trusted to generate high-entropy, single-use, non-memorable preimages when constructing the commitment.

Customers and permissionless callers: Invoice settlement, voucher creation, and post-expiry cancellation of both record types require no role. Whoever presents a valid PAS send request or an exact-amount coin can settle an open invoice, and any caller can trigger post-expiry cleanup, with voucher refunds always routed to the recorded customer. Customers are responsible for keeping their voucher preimage secret until redemption and for choosing a preimage with sufficient complexity to resist brute-force guessing.

Permissioned Asset Standard (PAS): The package builds all custody, transfer, mint, and burn behavior on PAS, which is outside the scope of this audit but inside its trust boundary. PAS is assumed to correctly enforce account custody, policy resolution, and its versioning controls. Two inherited dependencies warrant emphasis:

- The [pay](#) path resolves the customer's send request against the stablecoin's [Policy<Balance<C>>](#), which is configured and controlled by the stablecoin issuer. That issuer's compliance rules and version gating decide whether a settlement can resolve, so PAS-based invoice settlement assumes a live stablecoin policy that is permissive for the parties involved.

- Loyalty minting on settlement and voucher refunds on cancellation both terminate in a `deposit_balance` call on the customer's PAS `Account`. For an expired voucher, `cancel_voucher` is the only path that can extract the locked `Balance<LOYALTY>`, and it strictly depends on that deposit succeeding: if the customer's account ever reaches a state in which deposits abort (for example, through PAS governance or version invalidation), the cancellation aborts as a whole and the locked balance (along with the voucher entry's own storage deposit) remains stranded inside the `Merchant` with no alternative recovery path, since `redeem` is blocked after expiry and no privileged path can delete the voucher by discarding its balance. The system therefore assumes that PAS accounts remain able to accept deposits for as long as refunds may be owed.

Low Severity

L-01 Invoices and Vouchers Created with Wrong Parameters Cannot Be Canceled Before Expiration

Invoices are issued through the `create_invoice` function, which is gated by the `CashierRole`, and vouchers are created through the permissionless `create_voucher` function. In both cases the settlement-relevant fields are snapshotted at creation and become an `immutable commitment` for the record's lifetime: an invoice fixes its payout address, accepted payment type, line-item prices, amount, loyalty reward, and expiry, while a voucher fixes its loyalty-priced items, its locked `Balance<LOYALTY>`, its redeem commitment, and its expiry. Each record can be settled before expiry only through its own path, `pay` or `pay_with_coin` for an invoice, and `redeem` for a voucher, the latter additionally requiring the `CashierRole` and the customer's preimage. The dedicated cleanup paths, `cancel_invoice` and `cancel_voucher`, are permissionless but both abort with `ENotExpired` unless the current time is at or past the record's expiry.

As a result, neither an invoice nor a voucher created with wrong parameters can be invalidated before it expires. An invoice issued with an incorrect configuration (a wrong amount, incorrect line items, or a stale payout address) remains settleable at its incorrect snapshot for its entire lifetime, up to the seven-day maximum enforced by `MAX_TTL_MS`, so a customer who scans it may settle at the wrong terms, and the merchant has no on-chain recourse to prevent the settlement. The same happens with vouchers that lock customers' own `Balance<LOYALTY>`, and that balance cannot be reclaimed until the voucher expires and `cancel_voucher` becomes callable. The only pre-expiry exit for a voucher is `redeem`, which burns the locked balance rather than returning it, so it does not help a customer recover funds committed by mistake. In both cases the exposure or lock-up window is bounded by the configured time-to-live, but that value is fixed at issuance and cannot be shortened afterward.

Consider providing an authorized mechanism to invalidate an unexpired invoice or voucher, so that an erroneous invoice does not remain settleable, and a voucher's locked funds are not held, for the full time-to-live. The appropriate authority differs between the two: a cashier for invoices, and the owning customer for vouchers, since invalidating a voucher must return the locked `Balance<LOYALTY>` to the account that created it.

Update: Resolved in [pull request #45](#).

L-02 Unbounded Variants per Listing Can Make Pricing and Removal Exceed Transaction Limits

The merchant catalog stores each listing's variants in a `VecMap` embedded in the `Listing` value, alongside a reverse `variant_index table` in the `Merchant` that maps each variant to its listing. Variants are added one at a time through the `add_listing_variant` function, which is gated by the `CatalogManagerRole`. Neither the listing constructor nor the variant-insertion path enforces any maximum number of variants per listing.

This unbounded growth degrades two operations, both of which scale linearly with the number of variants on a listing. First, invoice and voucher issuance prices each line item by resolving its variant, which loads the entire listing and performs a `lookup over the embedded VecMap` that scans it twice, once for the `contains` check and again for the `get`. Second, the `remove_listing function` deletes one `variant_index` entry per variant, producing a linear number of dynamic-field removals in a single transaction. As a listing grows, issuance gas rises and, past Sui's per-transaction object-mutation and gas limits, `remove_listing` aborts entirely. At that point the listing can be removed only after being shrunk incrementally through repeated `remove_listing_variant` calls.

Because only the trusted `CatalogManagerRole` can add variants, this is not an externally exploitable condition and places no funds at risk. However, it is otherwise bounded only by Sui's object-size and per-transaction limits, so a sufficiently large listing can render its own issuance or single-transaction removal unusable.

Consider enforcing a maximum number of variants per listing, chosen conservatively enough to remain below Sui's per-transaction object-mutation and gas limits. A single such bound keeps line-item pricing inexpensive and keeps `remove_listing` executable within one transaction, addressing both concerns at once.

Update: Resolved in [pull request #43](#).

L-03 Reused Voucher Preimages Enable Cross-Voucher Redemption Replay

Voucher redemption is guarded by a hashlock. The `create_voucher function` stores a customer-supplied 32-byte `redeem_hash`, a `blake2b256` commitment to a secret, and

locks a `Balance<LOYALTY>` inside the voucher. The `redeem_function`, gated by the `CashierRole`, burns that locked balance once the caller supplies a `preimage` satisfying `blake2b256(preimage) == redeem_hash`. The preimage is passed as a plain transaction input, so it becomes publicly visible on-chain when the redeeming transaction executes. At issuance, `create_voucher` enforces only that `redeem_hash` is 32 bytes, but it does not require the commitment to be unique, nor does it bind the commitment to a specific voucher.

Because redemption discloses the preimage publicly and the commitment is not voucher-specific, a revealed preimage can be replayed against any other voucher that shares the same `redeem_hash`. If a customer or wallet reuses a secret across vouchers, redeeming one voucher exposes the preimage, after which the locked `LOYALTY` of every other voucher committed to the same hash can be redeemed without the customer's participation.

Consider maintaining a registry of `redeem_hash` commitments and rejecting, at issuance, any commitment that is already registered, so that no two vouchers can ever share a commitment. A commitment should be registered when its voucher is created. On cancellation it should be removed, since that path never discloses the preimage and the secret may safely be reused. On redemption, however, the commitment should be retained permanently because it reveals the preimage on-chain, so the same commitment must never again be accepted at creation. Alternatively, if this storage cost is considered prohibitive, document this limitation, making explicit that each voucher must use a fresh, single-use, high-entropy preimage and placing the responsibility for non-reuse on the client and wallet integrations that construct the commitment.

Update: Resolved in [pull request #42](#).

Notes & Additional Information

N-01 Configuration and Display Events Omit the Values They Change

The `ConfigUpdated` and `DisplayUpdated` events are emitted with no fields, even though `update_config` and `update_display` overwrite the merchant's `Config` and display in place. The previous `Config` (which carries the `payout_address` and accepted payment type) and the previous name and logo are dropped on replacement, and neither event records

the new or old values. The change history is therefore not recoverable from on-chain state or the event stream: on-chain state exposes only the current values, so a pure event consumer cannot distinguish a benign rate change from a `payout_address` rotation or an accepted-currency swap, and cannot audit these security-relevant administrative changes. The prior values do survive in the `update_config` and `update_display` transactions retained in historical checkpoints, but for an event-driven indexer that does not reconstruct full transaction history, the event stream is the only record it consumes for these overwrite-in-place changes, and in this case it is empty.

This is specific to replaced state. The terminal settlement events are self-contained, and the minimal `InvoiceCreated` and `VoucherCreated` events are sufficient, because an `Invoice` or `Voucher` is an immutable snapshot that remains fully readable via its emitted ID until it settles (resolving the ID on-chain is lossless). A replaced `Config` leaves nothing to resolve.

Consider emitting the changed fields on both events so that payout-address rotations, currency changes, and display updates can be reconstructed and monitored from the event stream alone.

Update: Resolved in [pull request #44](#).

N-02 Documentation Styling Inconsistencies

A few documentation-style inconsistencies remain across the package. None affects behavior or the accuracy of any individual comment, but each breaks the otherwise uniform documentation conventions and slightly raises the cost of reading the code:

- The ID-keyed view functions `invoice`, `voucher`, `invoice_receipt`, and `voucher_receipt` document only an `#### Aborts` section, while their catalog-side peers `listing`, `listing_variant`, and `active_listing_variant` include full `#### Parameters`, `#### Returns`, and `#### Aborts` sections.
- The `prune_invoice_receipts` and `prune_voucher_receipts` functions are `MerchantRole`-gated storage-reclamation operations, yet they sit under the `// === Settlement Functions ===` header rather than the `// === Admin Functions ===` header that holds their `MerchantRole` peers `update_config` and `update_display`. Relatedly, the settlement surface is split across two headers: `pay`, `pay_with_coin`, `cancel_invoice`, `create_voucher`, and `cancel_voucher` sit under `// === Public Functions ===`, while `create_invoice` and `redeem` sit

under `// === Settlement Functions ===`, even though `pay` and `pay_with_coin` are the stablecoin-settlement path.

- Within `receipt`'s `// === Package Functions ===` section, `new_payment`, `new_redemption`, and `destroy` are documented with plain prose (a single line for the two constructors, a short multi-line block for `destroy`), while their peers `new_item` and `compute_total` use the `#### Parameters` / `#### Returns` / `#### Aborts` sub-headers.

Consider aligning the view-function and package-function documentation depth (or codifying the prose-versus-sub-header convention explicitly), moving the two `prune_*` functions under `// === Admin Functions ===`, and optionally reconciling the settlement-versus-public section split so the header names match the module's own "full settlement surface" framing.

Update: Resolved in [pull request #47](#).

N-03 Redundant and Duplicated Code in `merchant` Module

The `merchant` module contains several constructs that are redundant, defensive, or duplicated:

- The `EZeroAmount` check in `create_voucher` is reachable but subsumed by the subsequent `EInvalidAmount` assertion. For any well-formed catalog with non-empty items and positive prices and quantities, a zero amount can never equal the items' positive total, so `EInvalidAmount` already rejects it.
- The `pay` and `pay_with_coin` functions share an almost verbatim validate-mint-receipt-emit body, including an identical nine-positional `receipt::new_payment` call, and differ mainly in fund routing, customer derivation, the trailing `paid_with_coin` flag, and one extra assertion in `pay` (`EWrongLoyaltyRecipient`, binding the loyalty account owner to the send-request sender) that `pay_with_coin` has no proven sender to enforce. Because the shared logic is copied rather than factored out, a change applied to one path can be forgotten in the other, producing divergent settlement semantics between the two flows.

Consider removing the `EZeroAmount` check because `EInvalidAmount` already handles the zero-amount case. For the `pay` and `pay_with_coin` duplication, consider extracting the shared settlement body into a private helper so the two paths cannot drift.

Update: Resolved in [pull request #46](#).

N-04 Incorrect Documentation

Throughout the package, several documentation comments describe the implementation incorrectly. While none of these has a runtime effect, each can mislead an integrator or auditor reading the code:

- The [module-level role map](#) and the [MerchantRole struct doc](#), both in `merchant.move`, list a `set_payout_address` function that does not exist; payout is rotated only through `update_config`. The same lists omit `prune_invoice_receipts` and `prune_voucher_receipts`, understating that `MerchantRole` can irreversibly delete stored receipt detail.
- The documentation on [new redeem unlock approval in loyalty.move](#) states that `redemption::create` is its only caller, but no such function exists; the sole caller is `merchant::create_voucher`.
- The [Variant.loyalty_price documentation](#) in `listing.move` attributes the `ENoLoyaltyPrice` abort to the `receipt` module, but that constant is defined in and raised by the `merchant` module.
- The [set_listing_status aborts documentation](#) in `merchant.move` attributes `EActiveStateUnchanged` to the `merchant` module, but it is defined in and raised by `listing::set_active`. Because both modules number their error codes from zero, the same numeric code denotes different conditions per module.
- The two-step deployment example in the [merchant module header](#) shows `config::new` and `merchant::create` calls whose parameters do not match the real signatures; a deployer copying the example would write code that does not compile and would be misled about where `payout_address` is supplied.
- The generics documentation for `pay` and `pay_with_coin` (and the struct-level documentation) states that the accepted-currency check runs against the live `config.accepted_payment_type`, but the `check` is performed against the currency snapshot on the invoice at issuance. The same misattribution appears in `config.move` itself: the [module header](#) and the [accepted_payment_type field documentation](#) both describe that field as the value used at runtime to reject `pay<C>` calls in the wrong currency, though at settlement the check reads the invoice's snapshot `payment_type`, not the live config.
- The [create_invoice aborts list](#) orders `ELengthMismatch` before `EItemsTooMany`, but the function body checks them in the opposite order, so an input violating both aborts with a different code than the documentation implies. The same list also places `EOrderRefTooLong` last though the body checks it before the catalog aborts. The divergence recurs in two further aborts lists: [create_voucher](#) documents `EBadHashLength` last though it is the fourth check and lists `EInvalidAmount` ahead

of the catalog aborts that actually run first, and `config::new` lists `EZeroVoucherTtl` before `ETtlTooLarge` though the invoice-TTL `ETtlTooLarge` check runs first.

- The `InvoiceCreated` and `VoucherCreated` field documentation in `events.move` describes the identifiers as IDs of newly shared objects, but neither `Invoice` nor `Voucher` is a Sui object; both are store-only structs held as values inside the `Merchant` tables and keyed by a fresh identifier. The `events` module header repeats the conflation at the module level, directing indexers to "resolve the embedded IDs back to the relevant objects" when those identifiers key store-only `Invoice`, `Voucher`, `Listing`, and `Variant` values rather than Sui objects.
- The inline overflow comments around `compute_loyalty` in `config.move` (the `MAX_DECIMALS` doc comment and the inline notes in the `compute_loyalty` body) misidentify the binding constraint. They cite a `u64` bound for a value that is a `u128` scale factor and omit that the `amount * coefficient` product sits near the `u128` ceiling and that the narrowing cast is safe only because the clamp comparison runs in `u128` first.
- The `Merchant.loyalty` field documentation says the bundle is "read via `loyalty()`," but no such `loyalty` accessor exists on `Merchant`; the bundle is never exposed whole, and only derived scalars are readable through `loyalty_treasury_cap_id`, `loyalty_policy_cap_id`, `loyalty_policy_id`, and `loyalty_supply`. The same comment names only `pay` (mint) and `redeem` (burn) as reaching the treasury cap, omitting that `pay_with_coin` mints as well.
- The `Listing.active` field documentation labels the flag a "display hint," but it is an enforced purchasability gate: `active_listing_variant` aborts with `EListingInactive`, so an inactive listing cannot be sold or redeemed against through `create_invoice` or `create_voucher`. The peer `active` accessor describes it correctly as "whether the listing is purchasable."
- The `Voucher.customer` field documentation calls the customer the "Recipient of the `Receipt` on `redeem`," but the redemption `Receipt` is stored in the merchant's `voucher_receipts` table, not transferred to the customer (`Receipt` is a store-only value with no `key`); the customer is only the receipt's attribution field. The "returned balance on `cancel`" half of the sentence is accurate.
- The `receipt` module header states that `Receipt<Payment>` "is built at the end of `merchant::pay`," naming only one of the two settlement paths; `pay_with_coin` also constructs and stores a `Receipt<Payment>`.
- The `config::new` documentation directs the caller to `merchant::create<C>`, but `merchant::create` is not generic and takes no type parameter. The `config` module header renders the update path as `merchant::update_config(new: Config)`, a

single-argument call, whereas the real signature is `update_config(&mut Merchant, &Auth<MerchantRole>, Config)` gated by `MerchantRole`, with the parameter named `config`.

Consider correcting each comment to reflect the implementation.

Update: Resolved in [pull request #51](#).

N-05 Incomplete Documentation

Several function-level documentation blocks omit behavior that a caller needs in order to use the function safely:

- The `create_voucher` [aborts list](#) omits aborts propagated from the customer-supplied `unlock_req`'s resolution. Unlike `pay`, `create_voucher` stamps the required `RedeemUnlockApproval` itself, so the exposure is not a missing customer approval but rather: the `approve` call aborts on a request that already carries that approval; and `unlock_funds::resolve` aborts on a stale PAS policy version, or when `unlock_req` carries any approval beyond the one this function adds (PAS requires the approval set to match the policy exactly). The same documentation also references a nonexistent `unlock_funds::resolve_balance` in the [policy_loyalty parameter description](#) where the real function is `resolve`.
- The `create_invoice` and `create_voucher` [aborts lists](#) both omit the `EAmountOverflow` abort propagated from `receipt::compute_total`, which each calls to total the priced line items. Because per-variant quantities are caller-supplied, a `quantity * price` product or running total exceeding the `u64` range is reachable through the public API.
- Neither `loyalty::create` nor `loyalty::mint_to` documents an `#### Aborts` section, though both propagate PAS aborts. `create` aborts if a `Policy<Balance<LOYALTY>>` already exists for the namespace (for example, if it is called a second time) or if the PAS version check fails; `mint_to` aborts on `LOYALTY` supply overflow or the receiving account's version check. The supply-overflow case is already documented on `mint_to`'s callers (`pay`, `pay_with_coin`) but not on the function itself.

Consider documenting on `create_voucher` the PAS-side aborts reachable when the customer-supplied `unlock_req` is resolved (the duplicate-approval abort from the `approve` call and the stale-version and extra-approval aborts propagated from `unlock_funds::resolve`) and correcting the `resolve_balance` reference to `resolve`

in the `policy_loyalty` parameter description; documenting the `EAmountOverflow` propagation on `create_invoice` and `create_voucher`; and adding `#### Aborts` sections to `loyalty::create` and `loyalty::mint_to`.

Update: Resolved in [pull request #50](#).

Conclusion

The `openzeppelin-sui-payments` package is a closed-loop point-of-sale system for Sui through which a merchant accepts a single permissioned stablecoin, rewards each settlement by minting a soulbound loyalty currency, and honors hash-locked vouchers that customers create to redeem accumulated loyalty for goods. Custody of both currencies is delegated to the Permissioned Asset Standard (PAS). OpenZeppelin audited the eight Move modules that compose the package, covering the shared merchant state and catalog, the invoice and voucher lifecycles, the loyalty minting and burning paths, the role-gated administrative surface, and the assumptions the package inherits from its PAS integration.

No issues of critical, high, or medium severity were identified. Three low-severity issues were raised, concentrated on lifecycle boundaries: invoices and vouchers issued with incorrect parameters cannot be invalidated before they expire; listings can accumulate enough variants for pricing and single-transaction removal to exceed Sui's per-transaction limits; and voucher commitments are not required to be unique, so a preimage revealed by one redemption can be replayed against any other voucher sharing the same commitment. The remaining findings are notes, most of which concern the accuracy and completeness of the in-code documentation, along with configuration and display events that omit the values they change and duplicated logic across the two payment settlement paths.

The codebase is compact and extensively documented, with a module layout that separates the stored data types from the business logic concentrated in the `merchant` module, and snapshot semantics that turn issued invoices and vouchers into binding commitments. Correcting the documentation inaccuracies collected in this report would keep those comments a reliable guide for integrators of the package. As reflected in the trust assumptions, the effective security of a deployment ultimately rests on the correctness and availability of the external PAS dependency, on responsible key management for the root and operational roles, and on wallets generating high-entropy, single-use voucher secrets, all of which sit outside the package's on-chain enforcement.

The OpenZeppelin development team is thanked for their collaboration and support throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.